TEMA 1 – Eventos y Formularios

> **Objetivo del tema**
>
> En este tema aprenderemos sobre los Eventos y los Formularios

1.1 Eventos

Consulta la presentación **UD05 - Eventos y Formularios**.

Un **evento** en JavaScript representa una acción o un suceso que **ocurre** en la página web o en el navegador y al cual el programa puede reaccionar ejecutando código. En general, podemos decir que un evento sucede cuando el usuario realiza "un cambio" sobre la página web.

El Manejador de Eventos es la **función** de JavaScript que permanece "escuchando" a que ocurra un evento específico sobre un objeto del DOM para ejecutarse inmediatamente después. Existen tres formas de asignar un manejador en el DOM:

1) **Mediante atributos HTML**, mezclando la lógica (JS) con la estructura (HTML). **Desaconsejado**.

```
<button onclick="alert('¡Hiciste clic!')">Aceptar</button>
```

A esto se le denomina **Modelo de Registro de Eventos en Línea** y plantea varios problemas. Para empezar, no separa programación de estructura. A demás, al añadir el evento como atributo del componente, se ejecuta primero el script de JS y después la acción por defecto del componente HTML.

Se entiende mejor con este ejemplo:

```
<a href="siguiente.html" onclick="alert('Click!')">Pulsa aquí</a>
```

Una posible solución es evitar que el navegador ejecute la acción por defecto:

```
<a href="siguiente.html" onclick="alert('Click!');
    return false;">Pulsa aquí</a>
```

2) **Mediante propiedades del DOM**. En este caso, tenemos separados el HTML y el JS. Lamentablemente, solo permite **un** solo manejador por evento. Si asignas otro, el anterior se sobrescribe.

Por ejemplo, dado un botón:

```
<button id="miBoton">Haz clic aquí</button>
```

Desde el JS podemos hacer:

```
const boton = document.getElementById("miBoton");

boton.onclick = function() {
  console.log("Primer manejador");
};
```

3) **Mediante addEventListener()**, que es a forma recomendada. Permite asociar **múltiples manejadores** al mismo evento sin sobrescribir los existentes y ofrece control preciso sobre las fases de propagación.

```
const boton = document.getElementById("miBoton");

// Escuchador 1
boton.addEventListener("click", function(event) {
  console.log("Evento registrado con addEventListener");
});

// Escuchador 2 (ambos se ejecutarán)
boton.addEventListener("click", (e) => {
  console.log("Elemento cliqueado:", e.target);
});
```

A esto se le denomina **Modelo de registro de eventos avanzado del W3C**.

1.2 Formularios

Un **formulario** es un elemento de la interfaz gráfica (<form> en HTML) compuesto por una colección de elementos (campos de texto, botones, etc.) cuya función principal es **recopilar datos** introducidos por el usuario para procesarlos, validarlos o enviarlos a un servidor o script.

Desde la perspectiva del DOM en JavaScript, un formulario consta de un contenedor, sus elementos, y un mecanismo de interacción, generalmente un botón.

Un ejemplo sencillo:

```
<form id="formContacto" name="contacto">
    <input type="text" name="usuario" placeholder="Tu nombre">
    <button type="submit">Enviar</button>
</form>
```

Desde JavaScript, podemos recoger el nombre del usuario cuando presiona el botón.

```
const miFormulario = document.forms["contacto"];
const campoNombre = miFormulario.elements["usuario"];
```

Action y method

Para entender el funcionamiento correcto de <form>, hay que mencionar dos de sus propiedades: Action y form. En el ejemplo de más arriba no aparecen. Esto se debe a que en los navegadores modernos se asumen sus valores por defecto si no están.

- **Si no pones action:** El navegador entiende que envías la información del formulario a la misma página/URL en la que estás. Puedes enviar esa información a donde quieras.

- **Si no pones method:** El navegador asume **GET** por defecto. El otro método es **POST**.

Un formulario genérico con la anotación completa tiene una estructura similar a esta:

```
<html>
    <head>
        <title>Mi formulario</title>
    </head>
    <body>
        <h3>Formulario</h3>
        <form action= "pagina.php" method="post">
            //Elementos
        </form>
    </body>
</html>
```

Mediante form.elements podemos acceder (desde JS) a todos los controles de entrada de un formulario, de forma que así podamos recuperar la información que contiene.

Selección de objetos de un formulario

Existen distintas formas desde JavaScript para obtener las referencias del DOM tanto del elemento <form> como de los controles que residen en su interior.

- form.elements [] - devuelve un array con todos los inputs de un formulario.
- getElementById () - devuelve un elemento que tenga un id determinado.
- getElementsByTagName() - devuelve un array de elementos de un tipo.

Dado el siguiente ejemplo:

```
<form id="formularioBusqueda" action="siguiente.php">
    <p>
        <input type="text" id="entrada" name="cEntrada">
        <input type="submit" id="enviar" name="enviar" value="Buscar...">
    </p>
</form>
```

Para referenciar al primer input podríamos hacer cualquiera de estas cosas:

```
document.getElementById("entrada")
document.formularioBusqueda.cEntrada
document.formularioBusqueda.elements[0]
document.forms["formularioBusqueda"].elements["cEntrada"]
document.forms["formularioBusqueda"].cEntrada
```

1.3 Descarga y ejecución de archivos .js

Cuando el navegador comienza a parsear un HTML, empieza línea a línea. Pero, a menos que se especifique lo contrario, en cuanto detecta que debe cargar un archivo JS, el parseo de detiene. Entonces, se descarga el fichero, ejecuta lo que tenga que ejecutar, y sólo entonces el parseo HTML continúa.

¿Ves el problema?

Es obvio: puede suceder que desde el JS se referencie a partes del HTML que aún no existan, y se generen errores. Para evitar esto:

- Atributo de etiqueta **script async**: Al establecer el atributo async, el script se descarga en paralelo al análisis de la página y se ejecuta tan pronto como esté disponible. El análisis de la página se interrumpe una vez que el script se descarga por completo, y lo ejecuta, y entonces continúa con el análisis del resto de la página. Obviamente, no siempre es una solución.

- Atributo de etiqueta **script defer**: Si se establece el atributo defer, se especifica que el script se descarga en paralelo al análisis de la página y se ejecuta después de que la página haya terminado de analizarse.

- Evento **DOMContentLoaded**: Si usamos la gestión de eventos con DOMContentLoaded sobre el documento, nos aseguraremos que el árbol DOM y la ventana están cargados.

```
document.addEventListener("DOMContentLoaded", () => {...});
```

- **Evento Load**: Se cargarán los JS Cuando todos los elementos de la ventana han sido cargados. Incluidas imágenes, subframes, estilos, etc.

```
window.addEventLIstener("Load", () => {...});
```

1.4 Cookies

Una **cookie** (o galleta informática) es un pequeño fragmento de texto o archivo de datos (normalmente no superior a 4 KB) que un servidor web envía al navegador del usuario y que este almacena localmente en su dispositivo.

Su propósito principal es **recordar información sobre la navegación del usuario** entre diferentes peticiones HTTP o entre distintas visitas a la misma página web. Por ejemplo, es muy interesante para aplicaciones tipo carrito de la compra.

Las cookies son específicas del dominio, de forma que un dominio no puede acceder a la cookie de otro dominio. Mientras realicemos solicitudes HTTP dentro de ese dominio, las cookies se enviarán de forma automática. Por último, recuerda que tienen una fecha de caducidad: el propio navegador se encarga de borrarlas cuando toca.

Las cookies no están exentas de fallas de seguridad:

- XSS(Cross-SiteScripting): Los atacantes pueden inyectar scripts maliciosos que roben cookies si éstas no están protegidas.
- CSRF(Cross-SiteRequestForgery): El navegador envía cookies automáticamente, lo que permite a un atacante ejecutar acciones no deseadas en sitios donde el usuario está autenticado.

Generalmente, las cookies se utilizan para:

1) Mantener las opciones de visualización.
2) Almacenar variables.
3) Seguimiento de la actividad del usuario.
4) Autenticación de usuarios.

Tienes un **ejemplo** del uso de cookies en su carpeta correspondiente. Para poder hacer que funcione correctamente, será necesario instalar el **plugin Live Server** a tu Visual Studio Code.

1.5 Web Storage

Web Storage es una API nativa de JavaScript que permite almacenar datos de tipo **clave-valor** directamente en el navegador del usuario de forma mucho más limpia, segura y con mayor capacidad que las cookies. Además, los datos en Web Storage no se envían automáticamente al servidor en cada solicitud HTTP.

… pero como le pasa a las cookies, no está exento de fallas de seguridad:

- XSS(Cross-SiteScripting): Los datos en Web Storage son accesibles desde cualquier script en la página. Si un atacante puede inyectar JavaScript malicioso, puede acceder a estos datos.

Por tanto, es recomendable evitar usarlo para almacenar información sensible, cifrarla, y borrarla cuando ya no se necesite.

Podemos almacenar la información de dos formas:

- **sessionStorage:** que permite almacenar información mientras el navegador está abierto; es decir, mientras dura la sesión de la página. Todas las páginas que tienen el mismo origen (dominio y protocolo) pueden acceder a estos datos indistintamente.

- localStorage: similar a sessionStorage, pero los datos se mantienen, aunque cerremos el navegador.

Tienes un **ejemplo** del uso de cookies en su carpeta correspondiente. Para poder hacer que funcione correctamente, será necesario instalar el **plugin Live Server** a tu Visual Studio Code.

Ejercicio Formulario I

Crea un **formulario** que recoja los siguientes datos referentes a una tienda de música. Elige el componente de formulario más adecuado para hacerlo (combos, radios, etc.):

- Nombre del disco.
- Grupo de música o cantante.
- Año de publicación.
- Tipo de música (podrá ser “rock”, “pop”, “punk” o “indie”);
- Localización: almacenará un número de estantería.
- Prestado: almacenará un valor booleano. Por defecto será false.

NOTA – Este ejercicio toma como base el ejercicio 7 del tema anterior.

Ejercicio Formulario II

Modifica el formulario diseñado en la actividad anterior para que los discos se inserten en el array de discos como nuevos objetos Disco (). Antes de introducirlo se deberá preguntar si se quiere añadir al comienzo o al final del array.

Ejercicio Eventos I

Observa el siguiente código HTML:

```
<body>
    <p id="contenidos_1">Lorem .....</p>
    <a id="enlace_1" href="#">Ocultar contenidos</a>
    <br/>
    <p id="contenidos_2">Lorem..... </p>
    <a id="enlace_2" href="#">Ocultar contenidos</a>
    <br/>
    <p id="contenidos_3">Lorem..... </p>
    <a id="enlace_3" href="#">Ocultar contenidos</a>
</body>
```

Realiza los siguientes cambios:

1. Cuando se pinche sobre el primer enlace, se oculte su sección relacionada.
2. Cuando se vuelva a pinchar sobre el mismo enlace, se muestre otra vez esa sección de contenidos.
3. Completar el resto de enlaces de la página para que su comportamiento sea idéntico al del primer enlace .
4. Cuando una sección se oculte, debe cambiar el mensaje del enlace asociado de ocultar a mostrar (pista: propiedad innerHTML).

Ejercicio Validación I

A partir del formulario en Ejercicio Formulario I y II, realiza los cambios necesarios para validarlo:

- Nombre del disco: 20 caracteres, obligatorio.
- Grupo de música o cantante: 20 caracteres, obligatorio.
- Año de publicación: 4 caracteres numéricos.
- Tipo de música (podrá ser “rock”, “pop”, “punk” o “indie”): sin comprobación.
- Localización: almacenará un número de estantería: vacío o numérico.
- Prestado: sin comprobación.

Tendrás que tener en cuenta, además:

- Que la validación se asignará al formulario mediante un AddEventListener.
- Que los campos nombre del disco y grupo de música se validarán en la misma función (campo20).

Ejercicio Validación II

A partir del formulario en Ejercicio Validación I, modifica las funciones de tal manera que, en caso de que se produzca un error en la validación, el campo implicado tenga el reborde rojo, y la etiqueta que lo acompaña también aparezca de color rojo.

Para ello deberás crear las clases css necesarias para que se visualice correctamente. Ten en cuenta que si el usuario mete correctamente el nombre, deberá volver a su color habitual.

Ejercicio Cookies

Vamos a añadir Cookies al formulario de Ejercicio Validación I. Crea un fichero que se llame “cookies.js” para ello. Tendremos que poder crear, borrar y consultar una cookie.

- Cada vez que el usuario trate de enviar el formulario y haya algún error, una variable contadora (almacenada en una cookie) se incrementará.
- El resultado del número de intentos se reflejará en un campo de texto que se encontrará al final del formulario.
- Si el usuario sale del programa y vuelve a entrar, el campo de texto mostrará el número almacenado en la cookie.
- Junto al campo de texto habrá un botón que, al pulsarlo, permitirá reiniciar el valor de la cookie a 0.

Ejercicio Cookies

Realiza una copia del Ejercicio Cookies. Elimina la parte de las Cookies, y crea un fichero que se llame “storage.js”. Debes poder hacer todo lo especificado en el ejercicio anterior pero con sessionStorage y localStorage.

Observa las diferencias en el comportamiento entre sessionStorage y localStorage